MVP را یک‌بار برای همیشه بفهمیم

فقط یک هدف: بدانید هر تابع «چرا» وجود دارد و «کجا» باید نوشته شود — بر اساس همان کد کلید شما

۱. مشکل اصلی: چه کسی چه چیزی را می‌داند؟

MVP یک «روش تقسیم کار» است، نه یک کتابخانه‌ی جادویی. فرض کنید یک رستوران دارید:

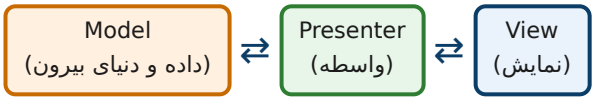
آشپزخانه = Model
غذا (داده) را می‌پزد و نگه می‌دارد. با دنیای بیرون (تأمین‌کننده = سخت‌افزار و FreeRTOS) در تماس است. **هرگز مشتری را نمی‌بیند.**

گارسون = Presenter
از آشپزخانه خبر می‌گیرد، تصمیم می‌گیرد چه چیزی به مشتری برسد و به سالن می‌گوید. **هم آشپزخانه را می‌شناسد هم سالن را.**

سالن = View
چیزی که مشتری می‌بیند: میز، منو، بشقاب (ویجت‌ها: متن، دکمه). **فقط دستور می‌گیرد، فکر نمی‌کند.**

قانون یگانه‌ی MVP

> هر کلاس فقط همسایه‌ی کناری خودش را می‌شناسد: View. **View ⇄ Presenter ⇄ Model** هرگز Model را نمی‌بیند و Model هرگز View را نمی‌بیند. اگر این را یادتان باشد، بقیه‌ی همه‌چیز نتیجه‌ی منطقی آن است.



چرا اصلاً این‌طور تقسیم می‌کنیم؟

- **تغییر ظاهر منطق را خراب نمی‌کند:** فردا بخواهید «Pressed» را به آیکون سبز تبدیل کنید، فقط View عوض می‌شود. Model و Presenter دست‌نخورده می‌مانند.
- **هر صفحه Presenter و View خودش را دارد** ولی **Model یکی است** (داده‌ی کل برنامه). پس Model نباید بداند الان کدام صفحه باز است.
- **TouchGFX فقط در تسک GUI امن است:** سخت‌افزار و RTOS فقط از یک دروازه (Model) وارد دنیای گرافیک می‌شوند.

۲. چرا باید «تابع تعریف کنیم»؟ — تابع = در بین دو اتاق

سؤال شما دقیقاً همین بود: «چرا مدام باید تابع بنویسم و نمی‌دانم کجا؟». جواب: چون دو کلاس که همدیگر را مستقیم نمی‌شناسند، فقط از طریق **تابع‌هایی که قبلاً اعلام شده‌اند** می‌توانند با هم صحبت کنند. هر تابع یک «در» است بین دو اتاق:

- کسی که تابع را **صدا می‌زند** (Caller) باید بداند چنین دری هست (یعنی در فایل hpp اعلام شده باشد).
- کسی که تابع را **پیاده‌سازی می‌کند** (Implementer) همان است که بدنه‌ی کار را می‌نویسد.

در پروژه‌ی کلید شما فقط **سه در** وجود دارد. هر در را ببینید که چه کسی صدا می‌زند و چه کسی پیاده می‌کند:

در (تابع)	کجا اعلام شد	چه کسی صدا می‌زند	چه کسی پیاده می‌کند	چرا وجود دارد
`modelListener->notifyButtonChanged(bool)`	`ModelListener.hpp`	Model	Presenter	Model نمی‌تواند Presenter را مستقیم بشناسد (در صفحه‌های مختلف فرق می‌کند)؛ پس یک «زنگ استاندارد» تعریف می‌کنیم که هر Presenter خودش جوابش را بدهد.
`view.updateButtonState(bool)`	`screenView.hpp`	Presenter	View	Presenter نباید به ویجت‌ها دست بزند؛ فقط می‌گوید «حالت را نشان بده» و View می‌داند چطور.
`osMessageQueueGet(...)`	`cmsis_os2.h` (آماده)	Model	FreeRTOS	تنها راه رسیدن داده از تسک دیگر به تسک GUI بدون تداخل.

`ModelListener` یعنی چه؟ — «لیست زنگ‌ها»

کلاس `ModelListener` فقط یک **فهرست از رویدادهایی است که Model می‌تواند اعلام کند**. هر خط `virtual ... {}` یک زنگ است. بدنه‌اش خالی است چون Model نمی‌داند اگر زنگ بخورد چه کسی چه کاری می‌کند. Presenter با `override` می‌گوید: «وقتی این زنگ خورد، من این کار را می‌کنم».

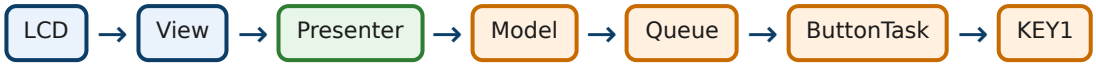
```
/* ModelListener.hpp — (شما می‌نویسید) فهرست زنگ‌ها */
virtual void notifyButtonChanged(bool buttonPressed) {}   // زنگ خالی

/* screenPresenter.cpp — (شما می‌نویسید) جواب زنگ */
void screenPresenter::notifyButtonChanged(bool pressed)   // override
{
    view.updateButtonState(pressed);                       // به سالن بگو
}
```

> **چرا نام باید دقیقاً یکی باشد؟** چون «override» یعنی «همان زنگ». اگر نام یا نوع پارامتر یک حرف فرق کند (مثل `notiy` در پروژه‌ی شما)، C++ آن را یک تابع تازه فرض می‌کند، زنگ اصلی بی‌جواب می‌ماند و هیچ خطایی هم نمی‌دهد. همیشه بنویسید `override` تا کامپایلر مچ‌گیری کند.

۳. یک رویداد را قدم‌به‌قدم دنبال کنیم

کلید فشرده می‌شود. این مسیری است که داده طی می‌کند؛ هر قدم نام فایل و صاحب آن را دارد. جمله‌ی «کدام تابع کجا؟» همیشه با این پرسش جواب داده می‌شود: **«این قدم در کدام اتاق اتفاق می‌افتد؟»**



۱ **ButtonTask کلید را می‌خواند و در صف می‌گذارد** (main.c — دنیای RTOS)

```
uint32_t state = HAL_GPIO_ReadPin(KEY1_GPIO_Port, KEY1_Pin);
osMessageQueuePut(QueueButtonHandle, &state, 0, 0);
```

این تسک فقط سخت‌افزار را می‌شناسد. از گرافیک خبر ندارد.

۲ **Model هر فریم صف را چک می‌کند** (Model.cpp — دروازه‌ی ورود داده)

```
while (osMessageQueueGet(QueueButtonHandle, &state, NULL, 0) == osOK)
```

`tick()` را خود TouchGFX هر ≈۱۶ms صدا می‌زند. `timeout=0` یعنی «اگر چیزی نبود منتظر نمان»؛ وگرنه صفحه فریز می‌شود. Model خام را به شکل ساده‌تر (`bool`) تبدیل می‌کند.

۳ **Model زنگ را می‌زند** (Model.cpp)

```
modelListener->notifyButtonChanged(buttonPressed);
```

Model نمی‌داند طرف مقابل کیست؛ فقط می‌داند «یک ModelListener هست». (خود TouchGFX قبلاً Presenter فعلی را با `model.bind(presenter)` در `modelListener` گذاشته.)

۴ **Presenter جواب زنگ را می‌دهد** (screenPresenter.cpp)

```
void screenPresenter::notifyButtonChanged(bool pressed) { view.updateButtonState(pressed); }
```

اینجا محل «تصمیم» است: می‌توانست بگوید «فقط اگر دما بالاست نشان بده» یا داده را تبدیل کند. در مثال ساده، مستقیم به View می‌دهد.

۵ **View ویجت را عوض می‌کند** (screenView.cpp)

```
Unicode::strncpy(textArea1Buffer, "Pressed", TEXTAREA1_SIZE);
textArea1.invalidate();
```

فقط View ویجت‌ها (`textArea1`) را می‌شناسد. `invalidate()` می‌گوید «این ناحیه را دوباره بکش»؛ بدون آن LCD عوض نمی‌شود.

> اگر هرگز یادتان نماند چه چیزی کجاست، این را به یاد بسپارید: **سخت‌افزار و RTOS → فقط تا Model. نام ویجت‌ها → فقط در View. تصمیم و تبدیل → Presenter.**

۴. هر فایل: «چه می‌داند، چه کار می‌کند، چه نمی‌داند»

برای تصمیم «این کد کجا برود»، فقط **فهرست چیزهایی را که کد استفاده می‌کند** نگاه کنید. اگر کد نام ویجت دارد، جایش View است؛ اگر `osMessageQueue` یا `HAL_` دارد، جایش Model (یا تسک) است.

فایل	می‌شناسد	وظیفه	هرگز در آن ننویس
`Model.cpp`	صف‌ها، HAL، RTOS، `modelListener`	گرفتن داده از دنیای بیرون، تبدیل ساده، زدن زنگ؛ و ارسال فرمان به سخت‌افزار	نام ویجت، `textArea1`، هر چیز گرافیکی `Unicode::`
`ModelListener.hpp`	فقط اعلام تابع‌ها	فهرست «زنگ‌ها» (virtual بدنه‌خالی)؛ نگهداری اشاره‌گر `model`	منطق یا پیاده‌سازی
`screenPresenter`	View (`view.`) و Model (`model->`)	override زنگ‌ها؛ تصمیم و تبدیل؛ انتقال دستور View به Model	دسترسی مستقیم به ویجت‌ها، `osMessageQueue`، `HAL_`
`screenView`	ویجت‌ها و `presenter`	تغییر ظاهر (متن، رنگ، نمایش/پنهان)؛ گزارش لمس به Presenter	منطق برنامه، `model->`، RTOS

«چه چیزی از کجا تولید می‌شود؟» (برای اینکه سرگردان نشوید)

تولیدشده توسط ابزار — دست نزنید	نوشته‌ی خودتان
پوشه‌ی `gui_generated/` (screenViewBase، `FrontendApplicationBase` (ساخت صفحه و `bind`)، `main.c` بخش‌های بین USER CODE، `QueueButtonHandle`	درون `gui/`: `Model.cpp/.hpp`، `ModelListener.hpp`، `screenView`، `screenPresenter`؛ و کد بین `USER CODE BEGIN/END` در `main.c`

«وصل کردن» را چه کسی انجام می‌دهد؟

شما هرگز `presenter`، `view` یا `modelListener` را با دست نمی‌سازید و وصل نمی‌کنید. وقتی صفحه باز می‌شود، فریم‌ورک این‌ها را انجام می‌دهد:

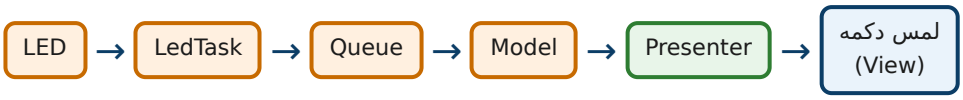
1. یک `screenView` و یک `screenPresenter` می‌سازد (حافظه‌ی استاتیک خودش).
2. `modelListener` → `model.bind(presenter)` در Model به Presenter اشاره می‌کند.
3. Presenter را به View و View را به Presenter متصل می‌کند (همان `view` و `presenter`).
4. `setupScreen()` و `activate()` را صدا می‌زند.

پس وقتی در Model می‌نویسید `modelListener->...`، آماده‌ی استفاده است. هنگام تغییر صفحه همه‌چیز نابود و برای صفحه‌ی جدید دوباره ساخته می‌شود؛ برای همین قبل از هر استفاده بنویسید `if (modelListener)`.

> **چرا صف؟** چون ButtonTask و GUI دو تسک جدا هستند و TouchGFX فقط در تسک GUI امن است. صف FreeRTOS یک صندوق نامه‌ی ایمن بین دو تسک است. ButtonTask می‌اندازد، Model در تسک GUI برمی‌دارد. هیچ‌کس مستقیم در کار دیگری دست نمی‌برد.

۵. جهت برعکس: وقتی کاربر چیزی را لمس می‌کند

تا اینجا داده از سخت‌افزار به صفحه می‌رفت. ولی MVP دوطرفه است. فرض کنید یک دکمه‌ی لمسی روی LCD می‌خواهید که LED را روشن/خاموش کند. قانون همان است؛ فقط جهت برعکس است و **درها همسایه به همسایه** هستند:



۱ **Designer:** روی دکمه Interaction بسازید: *Button clicked → Call new virtual function* با نام `ledButtonClicked`. Generate Code آن را در `screenViewBase` اعلام می‌کند و شما در View پیاده می‌کنید:

```
/* screenView.cpp */
void screenView::ledButtonClicked()
{
    presenter->ledToggleRequested();      // View فقط به Presenter می‌گوید
}
```

۲ **Presenter** (اعلام در hpp و پیاده در cpp):

```
void screenPresenter::ledToggleRequested()
{
    model->toggleLed();                   // Presenter به Model می‌گوید
}
```

۳ **Model** (اعلام در Model.hpp و پیاده در Model.cpp):

```
void Model::toggleLed()
{
#ifndef SIMULATOR
    uint32_t cmd = 1;
    osMessageQueuePut(QueueLedHandle, &cmd, 0, 0);   // timeout=0: GUI نباید معطل شود
#endif
}
```

۴ **تسک سخت‌افزار** (main.c) صف را با انتظار می‌خواند و پین را تغییر می‌دهد:

```
for(;;) {
  uint32_t cmd;
  if (osMessageQueueGet(QueueLedHandle, &cmd, NULL, osWaitForever) == osOK)
      HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
}
```

اینجا `osWaitForever` مشکلی ندارد چون این تسک LED است، نه GUI؛ هر وقت پیامی نیامد، فقط همین تسک می‌خوابد.

جهت	تابع «زنگ» چه کسی اعلام می‌کند؟	صدا زننده → پیاده‌کننده
داده به UI (بالا)	`ModelListener.hpp`	Model → Presenter → View
فرمان از UI (پایین)	`Presenter.hpp` (و `Model.hpp`)	View → Presenter → Model

این تقارن را حفظ کنید: **داده بالا می‌رود از طریق زنگ‌های ModelListener، و فرمان پایین می‌آید از طریق صدا زدن مستقیم توابع Presenter و Model.** چون Presenter هم `view` و هم `model` را دارد و View فقط `presenter` را.

۶. دستور ثابت برای هر برنامه‌ی جدید

برای هر مثال تازه، فقط این الگو را دنبال کنید و دیگر «کجا بنویسم؟» پیش نمی‌آید. مثال: نمایش دمای سنسور (عدد صحیح) روی یک `TextArea` با Wildcard به نام `tempText`.

#	فایل	چه می‌نویسید
۱	`CubeMX`	صف `QueueTemp` با Item Size برابر `uint32_t` + تسک `TempTask`
۲	`main.c` (تسک)	`uint32_t t = ReadSensor(); osMessageQueuePut(QueueTempHandle, &t, 0, 0); osDelay(500);`
۳	`ModelListener.hpp`	`virtual void notifyTempChanged(int t) {}` ← اعلام زنگ
۴	`Model.cpp`	`extern "C" osMessageQueueId_t QueueTempHandle;` و در `tick()` : Get با timeout=0 سپس `modelListener->notifyTempChanged((int)t)`
۵	`screenPresenter.hpp/.cpp`	`void notifyTempChanged(int t) override { view.updateTemp(t); }`
۶	`screenView.hpp/.cpp`	`void updateTemp(int t) { Unicode::snprintf(tempTextBuffer, TEMPTEXT_SIZE, "%d", t); tempText.invalidate(); }`
۷	`Designer`	TextArea با Wildcard (Buffer Size کافی) و Generate Code

ترتیب نوشتن را از «منبع داده» به «نمایش» بگیرید (۱ تا ۷)؛ هر قدم فقط به همسایه‌ی قبل و بعد خودش وابسته است.

سه پرسش برای هر کد جدید

1. **این کد با چه چیزی کار می‌کند؟** Model ← HAL/RTOS یا تسک؛ ویجت ← View؛ هیچ‌کدام (تصمیم/تبدیل) ← Presenter.
2. **داده بالا می‌رود یا فرمان پایین می‌آید؟** بالا ← زنگ در ModelListener؛ پایین ← تابع در Presenter/Model.
3. **آیا دو کلاس غیرهمسایه مستقیم با هم حرف می‌زنند؟** اگر بله (مثلاً Model → View)، اشتباه است؛ واسطه (Presenter) را وسط بگذارید.

خطاهای کلاسیک (همه در پروژه‌ی شما رخ داد)

خطا	اثر
اسم تابع در Presenter با ModelListener یکی نیست (`notiy`)	بدون خطا؛ هرگز اجرا نمی‌شود
handle اشتباه در `osMessageQueueGet` (`modelListener` به‌جای `QueueButtonHandle`)	بدون خطا؛ همیشه شکست
اندازه‌ی آیتم صف با نوع داده (۲ بایت در برابر ۴) نمی‌خواند	مقدار خراب یا ناقص
`osWaitForever` در `Model::tick()`	صفحه فریز می‌شود
فراموشی `invalidate()` یا Wildcard کوچک	LCD تغییر نمی‌کند یا متن بریده می‌شود

> **خلاصه‌ی یک‌خطی MVP:** Model داده را از دنیای بیرون می‌گیرد و «زنگ» می‌زند؛ Presenter تصمیم می‌گیرد؛ View فقط نمایش می‌دهد؛ و هیچ‌کس جز همسایه‌ی کناری‌اش را نمی‌شناسد. هر تابعی که می‌نویسید یکی از همین دَرها بین دو اتاق است.